

Aegis Exam — Project Viva Notes

Academic Examination & Remote Proctoring System

This document is the single reference for the viva. It maps the **Project Description (Phase 1)** → **Enhanced ERD (Phase 2)** → **SQL scripts and database objects (Phases 3–4)** so every claim in code can be defended against every claim in the description and the diagram.

1. Project at a Glance

A centralized SQL Server database for running secure, proctored academic exams at a university.

It covers four concerns:

1. **Academic structure** – University → Faculty → Department → Program → Course.
2. **Course delivery** – Semesters, course offerings, instructors, enrollments, waitlists, grades.
3. **Assessment** – Question banks, exams with sections, schedules, registrations, attempts, answers, results, score overrides.
4. **Integrity** – Proctoring telemetry (keystrokes, focus changes, clipboard), similarity comparisons, suspicious flags, disciplinary reviews, sanctions, appeals, appeal reviews.

Cross-cutting concerns: **Notification** (in-app alerts) and **Audit** (immutable change log).

The project is a Microsoft `.sqlproj` (SDK-style, target Sql170) — see [AegisExam.sqlproj](#).

2. Schema Layout

Defined in [Security/Schemas.sql](#). Each logical concern has its own SQL schema for namespacing and easier permissioning:

Schema	Purpose
[Identity]	People and their roles (Person, PersonRole, Student, Instructor, Admin)
[Academic]	Org structure + course delivery (University, Faculty, Department, Program, Semester, Course, CourseOffering, Enrollment, Waitlist, Grades, Rooms, Prerequisites)
[QuestionBank]	Banks, topics, questions, options, attachments
[Exam]	Exams, sections, exam-questions, schedules, rooms-on-schedule, invigilation, registrations, attempts, answers, score overrides, results, grade scale/band
[Proctoring]	Actions (and the 3 subtype tables), local action queue, similarity, suspicious flags
[Disciplinary]	Review → Sanction → Appeal → AppealReview
[Notification]	Outbound notifications to people
[Audit]	Immutable change/event log

Why schemas? It mirrors the **EER package boundaries** in the diagram (Identity_Person, Academic_Course, Exam_Exam, etc.). The dotted prefix in the ERD names is the schema name.

3. Entity-by-Entity Walkthrough

For every table: **purpose, key columns, constraints / business rules** baked into the DDL. File paths are clickable.

3.1 Identity Schema

Person

The supertype for every human in the system.

Column	Notes
PersonId	PK, IDENTITY
FirstName	NOT NULL
LastName	nullable
Email	UNIQUE, CHECK shape %_@%_._%_, max 254
PhoneNumber	UNIQUE, only 0-9 + () . - characters
DateOfBirth	between 1920 and today
NationalID	UNIQUE (13-char)
PasswordHash	hashed credential
CreatedAt	defaults GETDATE()

Why important: This is the **specialization root**. The ERD shows a **disjoint specialization (d)** from Person to Student / Instructor / Admin — meaning a Person can only be one of those three at one time in the role lookup, but technically the three subtype tables each FK back to Person and don't physically enforce disjointness. The logical disjointness lives in `[Identity].PersonRole`.

PersonRole

Lookup of roles a Person holds. Composite PK (`PersonId`, `RoleType`). CHECK restricts `RoleType` to `STUDENT | INSTRUCTOR | ADMIN`. Models the "has_role" relationship from the ERD.

Student

- FK to Person, FK to `Academic.Program` (every student is enrolled in exactly one program — matches description: "every student must be enrolled in one specific program").
- `RollNumber` UNIQUE, `EnrollmentYear` > 1900, `Status` ∈ {ACTIVE, GRADUATED, SUSPENDED, DROPPED}.

Instructor

- FK to Person and **Academic.Department** (departments "employ instructors").
- **Title**, **IsActive** (default 1).

Admin

- FK to Person. **RoleLevel** $\in \{\text{SUPER, DEPARTMENT, COURSE}\}$.
-

3.2 Academic Schema (Organisational + Delivery)

University

Top of the org tree. Holds country/timezone/website.

Faculty

FK to University. UNIQUE (**UniversityId**, **Code**) so two faculties in the same uni can't share a code.

Department

FK to Faculty. UNIQUE (**FacultyId**, **Code**). Matches description: *"Each faculty contains several departments."*

Program

FK to Department. **DegreeLevel** (BACHELOR/MASTER/etc.) and **TotalCreditHours** > 0 . Matches: *"Each department is responsible for offering various academic programs."*

Course

FK to Department. UNIQUE (**DeptId**, **Code**) (e.g., CS101 within CS dept). Matches: *"Departments maintain the catalog of courses."*

CoursePrerequisite

Composite PK (**CourseId**, **PrereqCourseId**). Both FKs point back to Course (self-relationship). **MinGrade** is the minimum letter grade a student needed in the prerequisite. Matches: *"A course can require other courses as prerequisites."*

Semester

FK to University. UNIQUE (**UniversityId**, **Name**). CHECK **EndDate** $>$ **StartDate**.

Room

Physical or virtual room. **IsVirtual** BIT (default 0). FK to University. Used by exams (**ExamRoom**).

CourseOffering

The fact table tying Course $\times$ Semester $\times$ Instructor. Has **Capacity** and **Status** (OPEN, CLOSED, etc.). Matches: *"During a semester, a department schedules specific courses as course offerings. Each course offering is taught by"*

a designated instructor."

Enrollment

Student × CourseOffering. UNIQUE (OfferingId, StudentId) prevents duplicate enrollments. Status tracks ENROLLED / DROPPED /

CourseWaitlist

Same key idea as Enrollment but for waitlist. Position is an integer queue position. UNIQUE (OfferingId, StudentId). Matches: *"If a course offering reaches its maximum capacity, students are placed on a waitlist."*

CourseEnrollmentGrade

1-to-1 with Enrollment (EnrollmentId is both PK and FK). LetterGrade, GradePoints. Matches: *"Upon completion of a course offering, the student receives a formal enrollment grade."*

3.3 QuestionBank Schema

QuestionBank

A department-scoped bank, created by an instructor. FK to Department and Instructor. Matches: *"Instructors author and manage questions within department-specific question banks."*

Topic

Organizes a bank into named topics. UNIQUE (BankId, Name). Matches: *"These banks are organized into specific academic topics."*

Question

The reusable question. FK to Topic and the authoring Instructor. Has:

- Type (MCQ, etc.), QuestionText (NVARCHAR(MAX))
- DefaultMarks >= 0, NegativeMarks <= 0 (CHECK constraints — negative marks are stored as ≤ 0 so the deduction is added)
- Difficulty, IsActive

QuestionOption

Multiple options per question. IsCorrect flags right answers. UNIQUE (QuestionId, OrderNum) keeps option order distinct.

QuestionAttachment

Files attached to a question (images, PDFs). Matches: *"Questions can have multiple options and file attachments."*

3.4 Exam Schema

GradeScale

A named grading scale (e.g., "Standard 4.0"), scoped to a university.

GradeBand

The bands inside a scale: letter grade + min/max percentage + GPA points. CHECK $0 \leq \text{MinPercentage} \leq \text{MaxPercentage} \leq 100$.

Exam

Created by an instructor for a course offering. Carries `ExamType`, `DurationMinutes > 0`, flags `IsRandomized`, `BrowserLockdown`, `OpenBook`, `NegativeMarking`, `Status`. FK to `GradeScale` used for final grading.

ExamSection

A section within an exam (e.g., "Section A"). UNIQUE (`ExamId`, `OrderNum`). Matches: *"An exam is structured into distinct sections."*

ExamQuestion

Joins `ExamSection` ↔ `Question` with order and per-exam marks override. UNIQUE on both (`SectionId`, `OrderNum`) and (`SectionId`, `QuestionId`) — no question repeats in a section, no two questions share an order number. Matches: *"Each section contains a specific sequence of questions pulled from the question bank."*

ExamSchedule

Date/time window + timezone for an exam.

ExamRoom

A specific room allocated to a specific schedule. `CapacityOverride` lets you reduce capacity below the room's real capacity.

InvigilationAssignment

Instructor assigned to an `ExamRoom` (i.e., to invigilate). Matches: *"Instructors are assigned to these rooms to supervise the exams."*

ExamRegistration

Student registers for an Exam. UNIQUE (`StudentId`, `ExamId`). `IsApproved` BIT and `Status`. Matches: *"Students must register for an exam to participate."*

Attempt

The actual sitting. FK to ExamRegistration (1 reg → many attempts possible). Records **StartTime**, **EndTime**, **IpAddress**, **BrowserInfo**, **OsInfo**, **Status**, **IsSubmitted**. FK to ExamRoom (where it was taken).

AttemptAnswer

Per-question response inside an attempt. Holds **SelectedOptionId** (for MCQs), **TextAnswer**, **FilePath**, **MarksAwarded**, **IsCorrect**, **GradedBy**, **GradedAt**. Matches: *"the student selects options or provides text answers ... These answers are then graded by an instructor or graded automatically."*

ScoreOverride

Admin-driven override of a final score. FK to **Identity.Admin**. Carries a **Reason** (required NVARCHAR(MAX)). Triggers a sync into the Result (see Trigger §6).

Result

1:1 with Attempt (**UNIQUE AttemptId**). Holds **FinalScore**, **Percentage**, **GradeBandId** (resolved at grading time), **IsPublished**, **PublishedAt**. Matches: *"producing a final calculated score and result for the attempt."*

3.5 Proctoring Schema

Action — the supertype

Every proctoring event tagged to an attempt. **ActionType** discriminator. **UNIQUE (AttemptId, SequenceNum)** so the client can deterministically order events.

The three subtype tables share **ActionId** as PK + FK (specialization implemented as the "**PK = FK**" pattern, mirroring the disjoint specialization in the ERD):

Subtype	Extra columns
KeystrokeAction	KeyPressed, ModifierKeys
FocusChangeAction	FromWindow, ToWindow, ProcessName
ClipboardAction	EventType (COPY/PASTE), ContentHash, ContentLength

Matches: *"the system continuously records proctoring actions, such as keystrokes, focus changes, and clipboard events."*

LocalActionQueue

Offline buffer for the proctoring agent. **ActionDataJson**, **QueuedAt**, **SyncedAt**, **IsSynced**, **RetryCount**. Lets the client push events even when temporarily offline.

SimilarityComparison

Pairwise comparison between two attempts. **UNIQUE (Attempt1Id, Attempt2Id)** plus **CHECK Attempt1Id < Attempt2Id** so each pair is stored exactly once (no (A,B) and (B,A) duplicates). **SimilarityScore**, **Method**, **IsFlagged**. Matches: *"The system compares different attempts to calculate similarity scores."*

SuspiciousFlag

A flag against an attempt. **Reason**, **FlaggedBy** (string — could be "ProctoringService" for the system or an actor), **Status**. Matches: *"automatically generates flags for suspicious behavior."*

3.6 Disciplinary Schema

Four-stage pipeline: **Review** → **Sanction** → **Appeal** → **AppealReview**.

Review

Instructor's review of an attempt. **Outcome** (GUILTY, WARNING, CLEARED, NO_ACTION, ...).

Sanction

Issued in response to a Review. **SanctionType** (ZERO_SCORE, WARNING, EXPULSION, ...). **IsActive** lets us deactivate it on an overturned appeal.

Appeal

Filed by a Student against a Sanction. **Status** (PENDING, UNDER_REVIEW, APPROVED, REJECTED, ...). **Reason** required.

AppealReview

The final adjudication record. **Decision** (UPHOLD, OVERTURN, PARTIAL, ...). FK to the reviewing instructor.

Matches: *"an instructor conducts a disciplinary review. This review may result in a formal sanction against the student. If a sanction is issued, the student has the right to file an appeal, which is subsequently reviewed by an administrator or instructor for a final decision."*

3.7 Notification & Audit

Notification

A message to a Person. **NotificationType**, **Channel** (EMAIL, SMS, IN_APP), **Subject**, **Body**, **SentAt**, **IsRead**.

AuditLog

Generic change log. **TableName**, **RecordId**, **ActionType** (INSERT, UPDATE, DELETE, PUBLISH, PUBLISH_REQUEST, ...), **OldValue**, **NewValue** (both NVARCHAR(MAX) so we can stuff JSON), **PerformedBy** (FK to Person, nullable for system actions), **PerformedAt**, **IpAddress**.

Defensive answer for viva — these two aren't named explicitly in the project description but are demanded *implicitly*: results being published, appeals being decided, and integrity flags being raised all need to (a) notify the student and (b) leave an audit trail. The triggers in §6 actually wire those flows together.

4. Mapping the Description ↔ ERD ↔ Schema

Business rule (from description)	ERD entity / relationship	Implementation
University consists of multiple faculties	Academic_University has_faculty (1:N) Academic_Faculty	Faculty.UniversityId FK + UNIQUE (UniversityId, Code)
Each faculty has departments	Academic_Faculty contains_dept (1:N) Academic_Department	Department.FacultyId FK
Departments offer programs	offers_program	Program.DeptId FK
Every student in exactly one program	enrolled_in_program (1:N)	Student.ProgramId FK NOT NULL
Departments own courses, employ instructors	owns_course, employs_instructor	Course.DeptId, Instructor.DeptId FKs
Course may require prerequisites	requires_prereq (recursive M:N)	CoursePrerequisite bridge table, self-referential
University runs semesters	runs_semester	Semester.UniversityId FK
Course offerings during a semester, taught by an instructor	schedules_offering, teaches, offered_as	CourseOffering(CourseId, SemesterId, InstructorId)
Students enroll, capacity → waitlist	makes_enrollment, has_enrollment, has_waitlist	Enrollment + CourseWaitlist; SP enforces capacity (\$5.1)
Final enrollment grade	yields_grade (1:1)	CourseEnrollmentGrade PK = FK on EnrollmentId
Department-specific question banks	maintains_bank / manages_bank	QuestionBank.DeptId, CreatedBy FKs
Banks → topics → questions → options/attachments	organizes_topic, categorizes_question, has_option, has_attachment	Topic, Question, QuestionOption, QuestionAttachment
Instructors create exams for offerings	creates_exam, offering_has_exam	Exam.OfferingId, Exam.CreatedBy FKs
Exam → sections → ordered questions	divided_into, section_contains, used_in	ExamSection, ExamQuestion
Schedules + rooms + invigilators	scheduled_in, allocated_to, room_used_in, supervised_by, invigilates	ExamSchedule, ExamRoom, InvigilationAssignment

Business rule (from description)	ERD entity / relationship	Implementation
Student registers, then attempts	<code>requires_reg</code> , <code>registers_for</code> , <code>generates_attempt</code> , <code>hosts_attempt</code>	<code>ExamRegistration</code> , <code>Attempt</code>
Per-question answer, graded	<code>has_answer</code> , <code>answered_in</code> , <code>selected_as</code> , <code>graded_by</code>	<code>AttemptAnswer</code>
Final score + result tied to a grade band	<code>produces_result</code> , <code>band_used_in</code>	<code>Result</code> (1:1 attempt), <code>Result.GradeBandId</code>
Proctoring telemetry subtypes	<code>records_action</code> + EER specialization <code>d</code> on <code>Proctoring_Action</code>	<code>Proctoring.Action</code> + 3 subtype tables (PK=FK pattern)
Local action buffering for offline clients	<code>buffers_queue</code>	<code>LocalActionQueue</code>
Similarity comparison + flags	<code>compared_in</code> , <code>triggers_flag</code>	<code>SimilarityComparison</code> , <code>SuspiciousFlag</code>
Disciplinary review → sanction → appeal → appeal review	<code>undergoes_review</code> , <code>results_in</code> , <code>challenged_by</code> , <code>resolved_by</code>	4 tables in <code>[Disciplinary]</code>
(implicit) notifications, audit trail	<code>receives_notif</code> , audit relations	<code>Notification.Notification</code> , <code>Audit.AuditLog</code> ; triggers fill these

Specialization (EER) details:

- `Person` → {`Student`, `Instructor`, `Admin`} — disjoint (the `d` in the ERD). Logical disjointness enforced by `[Identity].PersonRole` keyed on (`PersonId`, `RoleType`) (you can have multiple roles in theory but the lookup keeps it explicit; subtype tables hold role-specific attributes).
- `Action` → {`KeystrokeAction`, `FocusChangeAction`, `ClipboardAction`} — disjoint. Implemented with the **PK = FK** pattern: each subtype table's PK is also a FK to `Action.ActionId`.

5. Stored Procedures (Phase 4)

Five procedures, one per business workflow.

5.1 `[Academic].usp_EnrollStudentInCourseOffering`

File: `Academic/StoredProcedures/usp_EnrollStudentInCourseOffering.sql`

What it does: End-to-end enrollment with capacity handling.

Steps & guards:

1. `THROW 50001` if student missing or not `ACTIVE`.

2. Opens a transaction with **UPDLOCK**, **HOLDLOCK** on the offering row to prevent race-condition double-enrollments.
3. **THROW 50002** offering not found, **50003** not **OPEN**, **50004** already enrolled.
4. Counts current **Enrollment** rows where **Status <> 'DROPPED'** (also with **UPDLOCK**, **HOLDLOCK**).
5. If capacity reached → insert into **CourseWaitlist** with next position (returns **ActionTaken='WAITLISTED'**).
6. If **50005** already waitlisted, throws.
7. Otherwise inserts **Enrollment** with **Status='ENROLLED'** and returns **ActionTaken='ENROLLED'**.

Why two locks: Without **UPDLOCK + HOLDLOCK** two concurrent calls could each see "1 seat left" and both insert. Combined with the trigger in §6.1, accepting an enrollment also clears the waitlist row.

5.2 [Exam].usp_RegisterStudentForExam

File: [Exam/StoredProcedures/usp_RegisterStudentForExam.sql](#)

What it does: Atomically registers a student for an exam, only if they're enrolled in the underlying course offering.

Guards (THROW 50011–50016):

- Student must exist and be **ACTIVE**.
- Exam must exist and **Status ∈ {SCHEDULED, OPEN}**.
- An **Enrollment** row must exist for (student, exam's offering) with **Status='ENROLLED'**.
- At least one active **ExamSchedule** must exist for that exam.
- No duplicate **ExamRegistration**.

Returns the new **RegId**. **@AutoApprove** BIT defaults to 1.

5.3 [Exam].usp_PublishExamResult

File: [Exam/StoredProcedures/usp_PublishExamResult.sql](#)

What it does: Marks a **Result** as published and audits the action.

- **THROW 50021** if **@PublishedByPersonId** doesn't exist; **50022** if no **Result** exists for the attempt.
- Sets **IsPublished=1**, **PublishedAt=ISNULL(...,GETDATE())** (idempotent).
- Inserts an audit row **ActionType='PUBLISH_REQUEST'**.

The downstream **notification** to the student is handled by the trigger in §6.2 — separating "user action" (audit) from "side effect" (notify) keeps the procedure body simple.

5.4 [Proctoring].usp_RecordSuspiciousFlag

File: [Proctoring/StoredProcedures/usp_RecordSuspiciousFlag.sql](#)

What it does: Creates a **SuspiciousFlag** with **Status='OPEN'**.

- **THROW 50031** if attempt not found; **50032** if reason is blank/whitespace.
- **@FlaggedBy** defaults to **'ProctoringService'**.
- Returns the new **FlagId**. Trigger §6.4 then notifies the student and audits.

5.5 [Disciplinary].usp_SubmitAppeal

File: [Disciplinary/StoredProcedures/usp_SubmitAppeal.sql](#)

What it does: Student files an appeal against a sanction.

Guards (THROW 50041-50044):

- **Reason** must be non-empty.
- Sanction must exist and be **IsActive=1**.
- **Cross-table integrity:** verifies the student is actually the one tied to the sanction by walking **Sanction** → **Review** → **Attempt** → **ExamRegistration** → **StudentId**. This is the subtle business rule — you can't appeal *someone else's* sanction.
- No active appeal already exists (**PENDING** / **UNDER_REVIEW**).

Inserts **Appeal** with **Status='PENDING'**.

6. Triggers (Phase 4)

Five triggers, each wiring up a downstream side-effect to keep the data graph consistent.

6.1 [Academic].TR_Enrollment_AfterInsert_CleanupWaitlist

File: [Academic/Triggers/TR_Enrollment_AfterInsert_CleanupWaitlist.sql](#)

When a row is inserted into **Enrollment**:

1. Deletes any matching **CourseWaitlist** row for (**OfferingId**, **StudentId**) (so the same student isn't on the waitlist AND enrolled).
2. Re-numbers **Position** for everyone left on the waitlist for that offering, ordered by **Position ASC**, **RequestedAt ASC**, **WaitlistId ASC**, using a **ROW_NUMBER()** CTE. Keeps the queue dense (no holes).

6.2 [Exam].TR_Result_AfterInsertUpdate_PublishWorkflow

File: [Exam/Triggers/TR_Result_AfterInsertUpdate_PublishWorkflow.sql](#)

AFTER INSERT, UPDATE on **Result**:

1. Backfills **PublishedAt = GETDATE()** if a row is now **IsPublished=1** but has no timestamp.
2. Detects "newly published" results (was 0 or didn't exist, now 1) and sends an in-app **Notification** to the student, formatted with exam title + percentage.
3. Writes an **Audit.AuditLog** row with **ActionType='PUBLISH'**.

Uses two CTEs against **inserted/deleted** to detect the transition correctly.

6.3 [Exam].TR_ScoreOverride_AfterInsert_SyncResult

File: [Exam/Triggers/TR_ScoreOverride_AfterInsert_SyncResult.sql](#)

AFTER INSERT on **ScoreOverride**:

1. For each affected attempt, picks the **latest** override (`ROW_NUMBER() OVER PARTITION BY AttemptId ORDER BY OverriddenAt DESC, OverrideId DESC`).
2. Normalizes the new score to `[0, 100]`.
3. Updates `Result.FinalScore`, `Result.Percentage`, and re-resolves `Result.GradeBandId` from the exam's `GradeScale` via `OUTER APPLY` finding the band where the score falls between `MinPercentage` and `MaxPercentage`.
4. Audits the override.

This is the trigger that keeps Result coherent with the most recent admin override.

6.4 `[Proctoring].TR_SuspiciousFlag_AfterInsert_Audit`

File: [Proctoring/Triggers/TR_SuspiciousFlag_AfterInsert_Audit.sql](#)

`AFTER INSERT` on `SuspiciousFlag`:

1. Joins through `Attempt` → `ExamRegistration` → `Student` → `Person` to send the student an in-app notification (`PROCTORING_FLAG`).
2. Audits the flag insertion.

6.5 `[Disciplinary].TR_AppealReview_AfterInsert_DecisionWorkflow`

File: [Disciplinary/Triggers/TR_AppealReview_AfterInsert_DecisionWorkflow.sql](#)

`AFTER INSERT` on `AppealReview`:

1. Updates the parent `Appeal.Status` based on `Decision` (`OVERTURN`→`APPROVED`, `UPHOLD`→`REJECTED`, `PARTIAL`→`PARTIALLY_APPROVED`, else `UNDER_REVIEW`).
2. If `Decision='OVERTURN'`, deactivates the parent `Sanction` (`IsActive=0`).
3. Notifies the student.
4. Audits the decision, resolving the reviewer's instructor → person to attribute it.

7. Views (Phase 4)

Five denormalized read views — one per business "screen".

7.1 `[Academic].vw_CourseOfferingCapacity`

File: [Academic/Views/vw_CourseOfferingCapacity.sql](#)

Per offering: course code/name, semester, capacity, enrolled count (excluding `DROPPED`), waitlist count, `AvailableSeats`, computed `IsFull`, instructor info. Two CTEs (`EnrollmentStats`, `WaitlistStats`) feed a single SELECT. **Use case:** registrar / instructor dashboard.

7.2 `[Academic].vw_StudentProgramProfile`

File: [Academic/Views/vw_StudentProgramProfile.sql](#)

Flat one-row-per-student view joining `Student` → `Person` + `Program` → `Department` → `Faculty` → `University`. **Use case:** student profile lookup, ID card generation.

7.3 [Exam].vw_ExamResultLedger

File: Exam/Views/vw_ExamResultLedger.sql

Per result: exam info, attempt, recorded result, **latest** override (via `OUTER APPLY ... TOP (1) ORDER BY OverriddenAt DESC`), and `EffectiveFinalScore = COALESCE(override, recorded)`. **Use case:** result ledger / transcript export.

7.4 [Proctoring].vw_AttemptRiskOverview

File: Proctoring/Views/vw_AttemptRiskOverview.sql

Per attempt: action count, total/open flags, last flagged time, plus a computed `RiskLevel` (`HIGH` if open flags > 0 or total flags ≥ 3, `MEDIUM` if total = 2 or action count ≥ 15, else `LOW`). **Use case:** proctoring triage dashboard.

7.5 [Disciplinary].vw_AppealCaseOverview

File: Disciplinary/Views/vw_AppealCaseOverview.sql

Per appeal: sanction info, originating review, student, plus the **latest** `AppealReview` (again via `OUTER APPLY`). **Use case:** disciplinary committee work-queue.

8. Dummy Data Scripts

Two scripts, both included in the `.sqlproj`:

8.1 Scripts/Script.PostDeployment.sql

Idempotent `MERGE` statements — runs after every publish. **Heads-up for viva:** this file has a long-standing typo where many schemas are written as `[Identity.Student]` instead of `[Identity].Student`. If you run *just this* it will fail. The clean version is `DummyInserts.sql`.

Be honest if asked: `PostDeployment` is the *intent*, `DummyInserts` is what actually runs cleanly.

8.2 Scripts/Script.DummyInserts.sql

`SET IDENTITY_INSERT ... ON + INSERT INTO` blocks, schema by schema, in **dependency order** (parents before children) so all FKs resolve:

1. Academic core: University → Faculty → Department → Program
2. Identity: Person → PersonRole → Student / Instructor / Admin
3. Course catalog: Course → Semester → Room → CourseOffering → Enrollment → CourseEnrollmentGrade → CoursePrerequisite → CourseWaitlist
4. Grading scaffold: GradeScale → GradeBand
5. QuestionBank: QuestionBank → Topic → Question → QuestionOption → QuestionAttachment
6. Exam: Exam → ExamSection → ExamQuestion → ExamSchedule → ExamRoom → ExamRegistration → Attempt → AttemptAnswer → Result → ScoreOverride → InvigilationAssignment
7. Proctoring: Action → ClipboardAction / FocusChangeAction / KeystrokeAction → LocalActionQueue → SimilarityComparison → SuspiciousFlag

8. Disciplinary: Review → Sanction → Appeal → AppealReview

9. Notification, Audit

Each block toggles `IDENTITY_INSERT` on, inserts 1–5 rows, toggles it off.

9. Likely Viva Questions & Crisp Answers

Q: Why split into so many schemas? A: It mirrors the ERD packages and gives us natural namespaces — e.g. `[Exam].Exam` reads as "the Exam table in the Exam module". Easier permissioning later (you can `GRANT` on a schema).

Q: Where is generalization implemented? A: Two places. (1) `Identity.Person` is generalized into `Student`, `Instructor`, `Admin` — each subtype has its own table with `PersonId` FK back to `Person`; `PersonRole` enforces which roles a person holds. (2) `Proctoring.Action` is generalized into `KeystrokeAction`, `FocusChangeAction`, `ClipboardAction` using the **PK = FK pattern** — each subtype's PK is also a FK to `Action.ActionId`. Both are **disjoint** specializations (the `d` in the ERD).

Q: Why is `Result` 1:1 with `Attempt`? A: A single attempt produces exactly one final result. We enforce it with `UNIQUE(AttemptId)` on `Result`. Score overrides are separate (one attempt can have many overrides over time); the latest one is applied via `TR_ScoreOverride_AfterInsert_SyncResult`.

Q: How are similarity duplicates prevented? A: `CHK_SimilarityComparison_Order CHECK (Attempt1Id < Attempt2Id)` plus `UNIQUE(Attempt1Id, Attempt2Id)` — so we always store the pair in canonical order, never `(B, A)`.

Q: How is capacity enforced? A: Combination of the `Capacity` column on `CourseOffering` plus the stored procedure `usp_EnrollStudentInCourseOffering`, which counts non-DROPPED enrollments under `UPDLOCK`, `HOLDLOCK`, then either enrolls or waitlists. The trigger `TR_Enrollment_AfterInsert_CleanupWaitlist` then removes the student from the waitlist if they're enrolled directly.

Q: Where does negative marking live? A: Two levels. (1) Per-question: `Question.NegativeMarks` (`CHECK ≤ 0`). (2) Per-exam: `Exam.NegativeMarking` BIT — the exam can choose to honour question-level negative marks or not.

Q: Why do triggers also write to `AuditLog` and `Notification`? A: The description requires the system to be auditable and to keep the student informed. Putting the cross-cutting work in triggers means it happens automatically regardless of whether the row was inserted through a stored procedure, an ORM, or a manual SQL session.

Q: Is everything in the description actually modelled? A: Yes. The mapping table in §4 covers each business rule from the description to its ERD relationship and its physical implementation. Audit/Notification are inferred from "compliance" and "the student is notified" implications.

Q: What's a weakness of the design? A: Honestly: (1) `PostDeployment.sql` has bracket typos and would not deploy cleanly — `DummyInserts.sql` is the working one. (2) `Identity.Person` is a supertype but disjointness across `Student/Instructor/Admin` is enforced by convention (`PersonRole`) rather than by a hard DB constraint; the same `PersonId` could be inserted into multiple subtype tables. (3) Several `Status` columns use `VARCHAR` with no `CHECK` enumeration (`Enrollment.Status`, `CourseOffering.Status`, `Exam.Status`, `Appeal.Status`); we rely on application code. Worth fixing if asked.

10. Quick Index

- **Schemas:** [Security/Schemas.sql](#)
- **Project file:** [AegisExam.sqlproj](#)
- **Identity:** [Identity/Tables/](#)
- **Academic:** [Academic/Tables/](#), [Academic/Views/](#), [Academic/StoredProcedures/](#), [Academic/Triggers/](#)
- **QuestionBank:** [QuestionBank/Tables/](#)
- **Exam:** [Exam/Tables/](#), [Exam/Views/](#), [Exam/StoredProcedures/](#), [Exam/Triggers/](#)
- **Proctoring:** [Proctoring/Tables/](#), [Proctoring/Views/](#), [Proctoring/StoredProcedures/](#), [Proctoring/Triggers/](#)
- **Disciplinary:** [Disciplinary/Tables/](#), [Disciplinary/Views/](#), [Disciplinary/StoredProcedures/](#), [Disciplinary/Triggers/](#)
- **Notification:** [Notification/Tables/Notification.sql](#)
- **Audit:** [Audit/Tables/AuditLog.sql](#)
- **Dummy data:** [Scripts/Script.DummyInserts.sql](#)